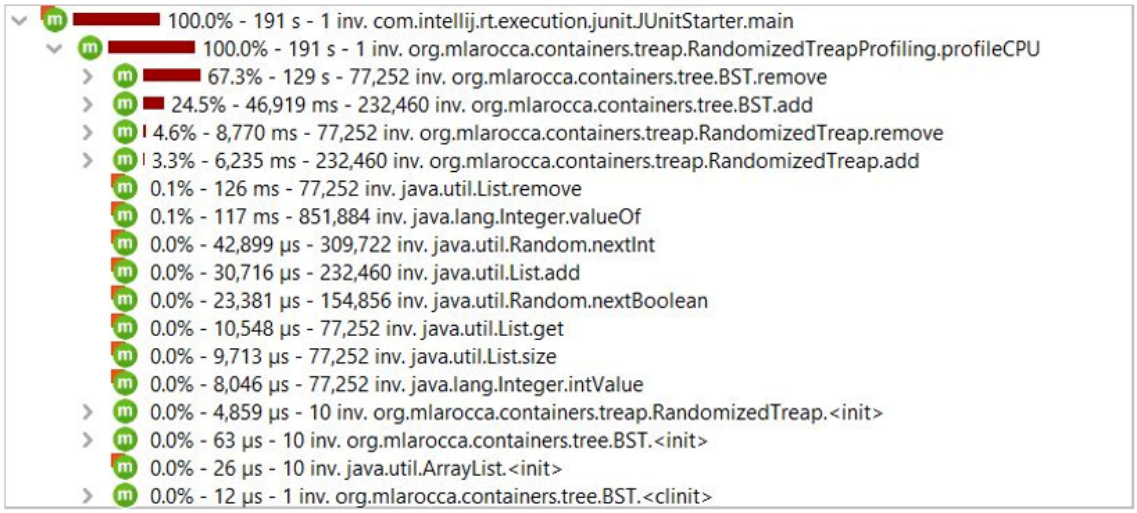


Figure 3.21 Profiling CPU usage when inserting/removing random integers between 0 and 1000 in a BST and randomized treap.

Figure 3.21 shows the result of this profiling: we can immediately see that the situation has radically changed, and BSTs perform tremendously worse now. So much worse that `BST::add` takes 8-fold the running time of `RandomizedTreap::add`, and for method remove the ratio is even worse, we are talking about almost a 15-fold speed-up using randomized treaps: that's exactly the situation where we would want to use our new, fancy, balanced data structure!

For the sake of completeness, let's also take a look at the worst-case scenario for BSTs: figure 3.22 shows the profiling of the last test case introduced in section 3.5.2, where we insert an ordered sequence in our containers. I believe that the results, in this case, don't even need to be commented, as we are talking about thousands of seconds versus microseconds (we had to test on a smaller set because BSTs performance was so degraded to be unbearably slow).